Aircraft Maintenance Anomaly Detection System

Presented to

Dr. Zhaoxian Zhou

In Partial Fulfillment of the Requirements for

CE 430 Senior Project

Written by

Pratima Rajbanshi (*w10161681*), Computer Science

Samarpan Thapa (*w10163103*), Computer Science

School of Computing Sciences and Computer Engineering

University of Southern Mississippi

Date Last Edited: March 13, 2026

Deadline of Submission: 03/13/2026

Scoring Sheet Based on Preset Rubrics

Section	**Scoring**
0: Abstract (5%)	
1: Introduction (5%)	
2: Review (5%)	
3: Engineering Standards (5%)	
4: Methodology (15%)	
5: Design Contraints (5%)	
6: Design/Imlementation (20%)	
7: Results/Evaluation (20%)	
8: Discussion/Conclusion (5%)	
9: References (5%)	
10: Writing (5%)	
11: Originality (5%)	
12: Bonus (5%)	
OVERALL (100%)	

Contents

Abstract **4**

Acknowledgments **5**

List of Figures **6**

List of Tables **7**

1 Introduction **8**
1.1 Background 8
1.2 Problem Statement 8
1.3 Project Objectives 8
1.4 Scope 9

2 Literature Review **10**
2.1 Overview of Related Work 10
2.2 Previous Approaches 10
2.3 Gaps and Relevance 10

3 Engineering Standards **11**
3.1 Industry Standards 11
3.2 Compliance and Safety 11

4 System Design and Methodology **12**
4.1 System Overview 12
4.2 Software Architecture and Tools 12
4.3 Development Methodology 13
4.4 System Flow 13

5 Design Constraints **14**
5.1 Technical Constraints 14
5.2 Resource and Performance Constraints 14
5.3 Trade-offs 14

6 Implementation Progress **15**
6.1 Completed Work 15
6.2 Challenges 15

7 Preliminary Results **16**
7.1 Anomaly Detection Performance 16
7.2 Observations 16

8 Current Status and Remaining Work **18**
8.1 Current Status 18
8.2 Remaining Work 18
8.3 Timeline 18

Bibliography **18**

Abstract

The project proposes the development of a system for anomaly detection and Remaining Useful Life (RUL) prediction for landing gear sensor data of aircraft. Six different anomaly detection algorithms, namely, Z-Score, Isolation Forest, K-Mean, DBSCAN, One-Class SVM, and LSTM Autoencoders, are implemented for six different synthetic Airbus datasets, consisting of 278,000 data points. An interactive dashboard using the Streamlit library facilitates the results for the maintenance crew. So far, the implementation of all the anomaly detection algorithms and the dashboard have been completed. Preliminary results indicate the feasibility of the Isolation Forest algorithm. Further, cross-validation of the results over different datasets, tuning of the parameters, and testing of the results remain to be done.

Acknowledgments

We would like to thank Dr. Zhaoxian Zhou for his guidance throughout the project and UWE Bristol for their publicly available synthetic landing gear datasets. We would also like to thank the University of Southern Mississippi School of Computing Sciences and Computer Engineering for providing the resources to complete this capstone.

List of Figures

4.1 System Architecture 12

7.1 Detection Rate and Computation Time Comparison (Dataset 1) 17
7.2 Bogie Pitch Trimmer Pressure with Isolation Forest Anomaly Detection (Set 0) 17

List of Tables

7.1 Anomaly Detection Performance (Dataset 1: Bogie Pitch Trimmer, 40,000 rows, 8 sensors). 16

8.1 Remaining Project Timeline 18

Chapter 1

Introduction

The current state of aircraft maintenance involves massive amounts of data from various sensors. Scheduled maintenance reduce the occurrence of failures, but early signs of failures, e.g., minor pressure changes, temperature surges, or slow degradation, are hidden amongst thousands of other noises and very hard to identify by human beings. [2, 3]. This project proposes the application of data-driven anomaly detection and RUL estimation for synthetic landing gear sensor data, highlighting the role of machine learning as a supporting system. [14].

1.1 Background

The landing gear system on an airplane is a safety-critical system that endures tremendous mechanical stress under flight conditions such as takeoff and landing [5]. With the use of data analytics, predictive maintenance can be achieved to avoid failures before they happen [4]. However, there is a lack of publicly available data for landing gear. So, this project usees the synthetic datasets provided by UWE Bristol [18], which are based on real Airbus data and created with GANs [7].

1.2 Problem Statement

The data related to aircraft maintenance is highly voluminous, complex, and distributed over time. The current techniques are unable to detect small changes that often occur prior to the degradation of the equipment. The present project aims to develop an automatic system that can analyze the landing gear sensor data and identify abnormal patterns, yet be understandable to maintenance personnel.

1.3 Project Objectives

1. Implement and compare six anomaly detection algorithms on landing gear time-series data.

2. Develop an RUL estimation module for components exhibiting gradual degradation.

3. Build an interactive dashboard for data exploration, parameter configuration, and anomaly inspection.

4. Evaluate each method's detection rate and computational performance across six datasets.

1.4 Scope

In scope: Six synthetic CSV datasets; six anomaly detection methods; RUL estimation; Streamlit dashboard; method comparison.

Out of scope: Real-time streaming; integration with aircraft maintenance management systems; autonomous decision-making.

Chapter 2

Literature Review

2.1 Overview of Related Work

A detailed taxonomy of anomaly detection methodologies is given in Chandola et al. [3]. The most promising approaches are ensemble methods and deep learning for predictive maintenance, as indicated in Carvalho et al. [2]. The most popular RUL dataset used in research is the NASA C-MAPSS dataset [15]. However, it only considers landing gear failure in engines. The linear degradation model is still effective for sensors with a monotonous trend, as indicated in Si et al. [17].

2.2 Previous Approaches

Statistical Methods (Z-Score) enable fast anomaly flagging but assume normality and ignore multivariate dependencies [3]. **Isolation Forest** [11] uses anomaly isolation based on random partitioning, where the computational complexity is linear. **K-Means** and **DBSCAN** [6] use clustering for anomaly detection, where the number of clusters is not required for DBSCAN. **One-Class SVM** [16] learns a boundary around normal data but is computationally expensive. **LSTM Autoencoders** [12] reconstruct normal sequences and flag high-error sequences as anomalous, capturing temporal dependencies that point-level methods miss [8].

2.3 Gaps and Relevance

The gaps in the existing research include the scarcity of landing gear-related data, the absence of comparative studies using multiple methods, the lack of interpretability of the results in deep learning models, and the absence of user-centric tools. [18]. This project attempts to bridge all four gaps by comparing six methods on the same landing gear datasets and developing an interactive dashboard. We use scikit-learn [13] and TensorFlow [1] as implementation frameworks.

Chapter 3

Engineering Standards

3.1 Industry Standards

The current project is based on **IEEE/ISO/IEC 12207** [9] standards for software life cycle processes and **IEEE/ISO/IEC 29148** [10] for requirements engineering. The code is written in accordance with PEP 8 guidelines and uses type hints and modular design. It is also version-controlled using Git.

3.2 Compliance and Safety

The datasets are used under Creative Commons licenses with proper attribution to UWE Bristol. The system is clearly designed to be a decision support tool, detecting anomalies and highlighting them, but not taking any maintenance actions. It does this to follow aviation industry standards. There are multiple methods to detect anomalies with variable thresholds, which can be adjusted to suit the user's needs, along with transparency in the dashboard.

Chapter 4

System Design and Methodology

4.1 System Overview

The system is a Python-based analytics pipeline with four components: (1) data ingestion and preprocessing, (2) anomaly detection algorithms, (3) RUL estimation, and (4) an interactive visualization dashboard. Figure 4.1 shows the architecture.

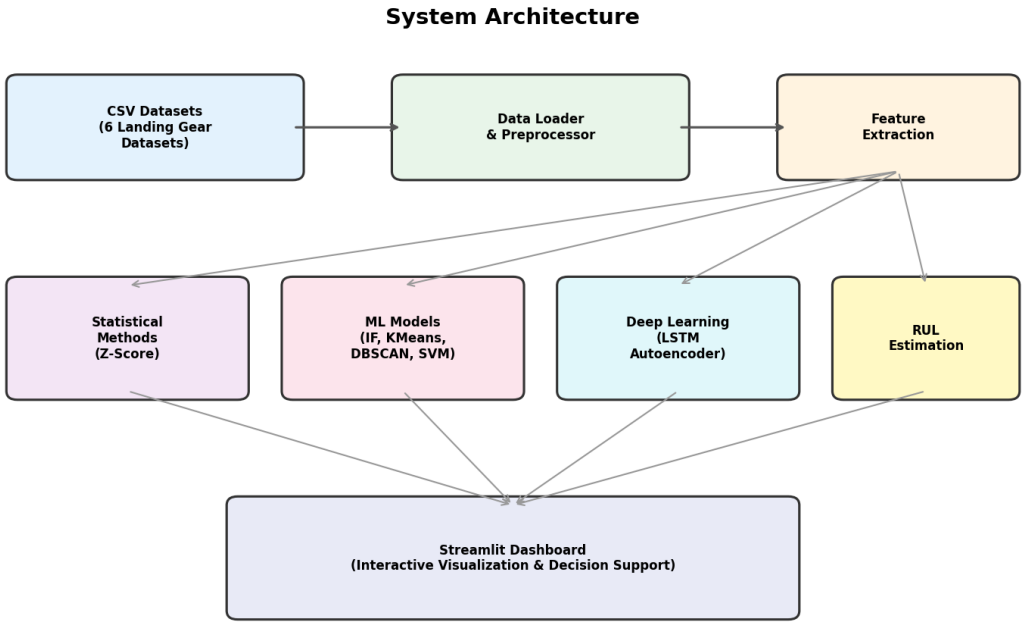

Figure 4.1: System Architecture

4.2 Software Architecture and Tools

The software consists of four Python modules: `data_loader.py` (CSV loading, normalization, statistics), `anomaly_detection.py` (five detection methods plus RUL), `autoencoder.py`

(LSTM encoder-decoder), and `visualization.py` (Plotly charts). The Streamlit dashboard (`app.py`) integrates all modules into five pages: Overview, Data Explorer, RUL Estimation, Anomaly Detection, and LSTM Autoencoder.

Key technologies: Python 3.13, pandas/NumPy, scikit-learn [13] (Isolation Forest, K-Means, DBSCAN, One-Class SVM), TensorFlow/Keras [1] (LSTM Autoencoder), SciPy, Plotly, and Streamlit.

4.3 Development Methodology

Development followed three iterative phases: (1) data exploration and summary statistics, (2) modular algorithm implementation with configurable parameters, and (3) dashboard integration with Streamlit caching for performance.

4.4 System Flow

The user can choose a dataset and method through the sidebar. The data loader will return a pandas DataFrame, the detection algorithm will add `is_anomaly` flags, and the visualization module will render interactive charts. For RUL estimation, linear regression will extrapolate degradation trends to critical thresholds.

Chapter 5

Design Constraints

5.1 Technical Constraints

The synthetic datasets may not capture the failure modes present in the real world. There are no ground truth anomaly labels available, so supervised learning is not feasible. There are six datasets, ranging from 20,000 to 108,000 rows, and 2 to 24 sensors. Therefore, the methods must generalize well for different dimensionalities.

5.2 Resource and Performance Constraints

The project is using open-source tools, and the license cost for these tools is zero. These tools are being executed on consumer-grade hardware, and the size of the deep learning model must be limited. Therefore, the methods must execute the datasets in a matter of seconds. Z-Score takes less than 0.01 s, and Isolation Forest takes 0.56 s, whereas DBSCAN takes 11.98 s and One-Class SVM takes 13.45 s. Therefore, these two algorithms qualify for the project, whereas the other two algorithms are slow. Since the project is using the Streamlit framework, the data loading would be cached, so there would be no redundancy.

5.3 Trade-offs

The project needs to consider the following trade-offs: speed and accuracy, simplicity and power, sensitivity and specificity. Z-Score is the fastest but only detects extreme univariate outliers. Isolation Forest is fast and captures multivariate anomalies. K-Means and DBSCAN can capture complex patterns but are slower and require parameter tuning. One-Class SVM is powerful but computationally expensive. LSTM Autoencoders can capture temporal dependencies but require more data and tuning.

Chapter 6

Implementation Progress

6.1 Completed Work

The program consists of about 600 lines of Python code in four modules. The `data_loader.py` module loads all six datasets, extracts the sensor columns, normalizes the features, and calculates the statistics. The `anomaly_detection.py` module includes the implementation of the Z-Score method, Isolation Forest, K-Means, DBSCAN, and One-Class SVM algorithms, as well as the RUL estimation using linear extrapolation. The `autoencoder.py` module includes the implementation of the autoencoder algorithm using a two-layer LSTM encoder-decoder architecture that detects anomalies based on the reconstruction error. The `visualization.py` module provides six different types of Plotly charts. The program is connected with the Streamlit dashboard.

6.2 Challenges

The challenges encountered in the project include the variety of the datasets, which makes the program require flexible code for different sizes and names of the columns. The DBSCAN algorithm is extremely sensitive to the choice of the parameter epsilon. The training time of the LSTM autoencoder is relatively long. The inability to compute precision/recall without ground-truth labels.

Chapter 7

Preliminary Results

7.1 Anomaly Detection Performance

We compare the performance of the methods using the detection rate, computation time, and qualitative assessment of the domain. Table 7.1 shows the results for the methods on Dataset 1.

Table 7.1: Anomaly Detection Performance (Dataset 1: Bogie Pitch Trimmer, 40,000 rows, 8 sensors).

Method	Anomalies	Rate (%)	Time (s)
Z-Score ($\theta = 3.0$)	1,327	3.32	0.01
Isolation Forest ($c = 0.05$)	2,000	5.00	0.56
K-Means ($k = 3$, $p = 95$)	2,000	5.00	2.16
DBSCAN ($\varepsilon = 1.5$, $m = 10$)	4	0.01	11.98
One-Class SVM ($\nu = 0.05$)	2,000	5.00	13.45

7.2 Observations

The best trade-off of detection quality (5%) and efficiency (0.56s) is achieved by the Isolation Forest algorithm. The Z-Score method is the fastest (0.01s) but detects only extreme univariate outliers (3.32%). The DBSCAN method detects only 4 points, and hence its parameters need to be tuned for the given data. The One-Class SVM method performs as well as the Isolation Forest method but is 24 times slower. These observations are consistent with the expectations based on the literature [11, 6, 16]. Figure 7.1 and Figure 7.2 show the method comparison and the sample detection results.

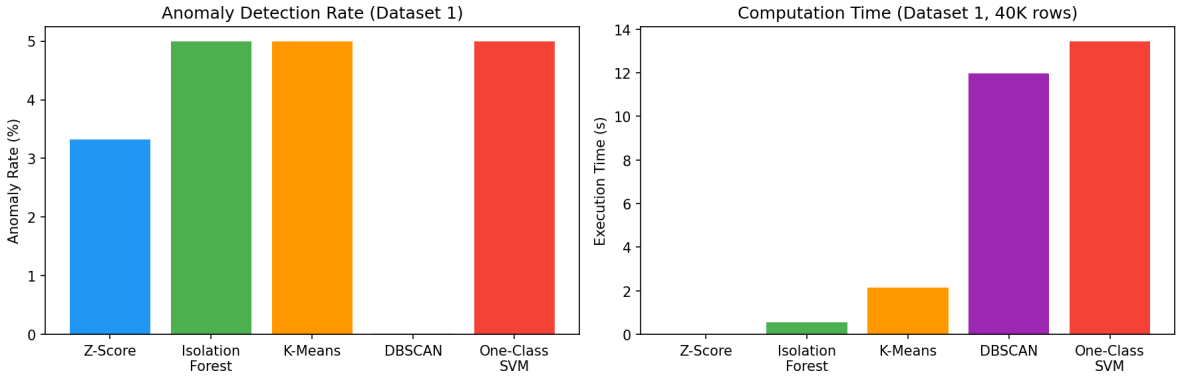

Figure 7.1: Detection Rate and Computation Time Comparison (Dataset 1)

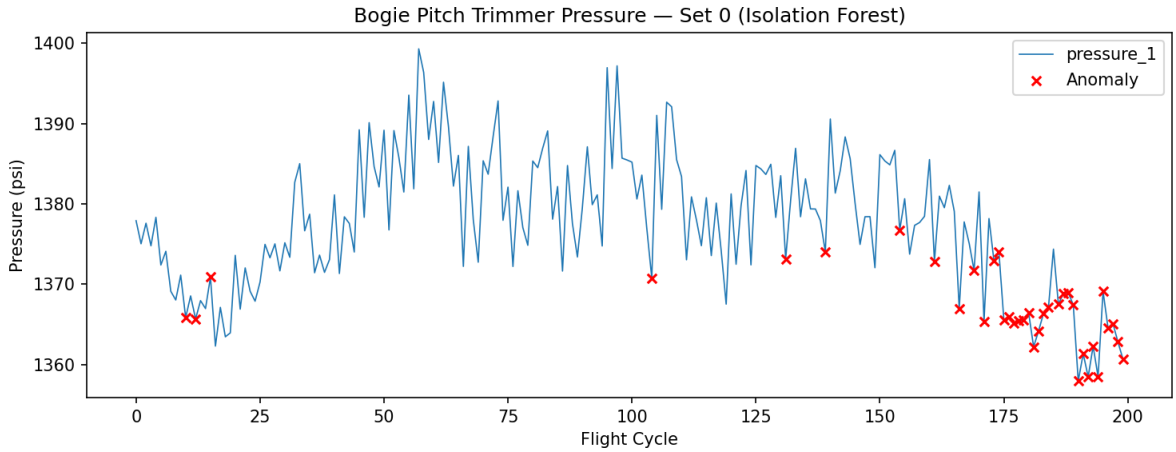

Figure 7.2: Bogie Pitch Trimmer Pressure with Isolation Forest Anomaly Detection (Set 0)

Chapter 8

Current Status and Remaining Work

8.1 Current Status

The following tasks are completed as of the midterm: - Loading and validation of all six datasets. - Implementation of five-point level detection algorithms. - LSTM autoencoder is working with the default hyperparameters. - RUL estimation is functional across all six datasets. - Streamlit dashboard is operational. - Preliminary comparison on Dataset 1 is complete.

8.2 Remaining Work

1. Cross-dataset evaluation across all six datasets.
2. LSTM Autoencoder hyperparameter tuning.
3. DBSCAN automated parameter selection.
4. RUL validation across more time series.
5. Dashboard polish and comprehensive testing.
6. Final report with full analysis and conclusions.

8.3 Timeline

Table 8.1: Remaining Project Timeline

Week	Tasks
Week 9–10	Cross-dataset evaluation, LSTM tuning
Week 11–12	DBSCAN optimization, RUL validation
Week 13–14	Dashboard polish, comprehensive testing
Week 15–16	Final report writing and submission

Bibliography

[1] M. Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems. `https://www.tensorflow.org/`, 2015. Software available from tensorflow.org.

[2] T. P. Carvalho, F. A. A. N. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, and S. G. S. Alcalá. A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering*, 137:106024, 2019.

[3] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

[4] M. Compare, P. Baraldi, and E. Zio. Challenges to IoT-enabled predictive maintenance for industry 4.0. *IEEE Internet of Things Journal*, 7(5):4585–4597, 2020.

[5] M.-H. Do and N. Vichare. Proactive condition-based maintenance for aircraft landing gear. *Reliability Engineering & System Safety*, 87:155–165, 2004.

[6] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231, 1996.

[7] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2014.

[8] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[9] IEEE. IEEE/ISO/IEC 12207-2017: Systems and software engineering – software life cycle processes. *IEEE Standard*, 2017.

[10] IEEE. IEEE/ISO/IEC 29148-2018: Systems and software engineering – life cycle processes – requirements engineering. *IEEE Standard*, 2018.

[11] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008.

[12] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*, 2016.

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[14] Y. Ran, X. Zhou, P. Lin, Y. Wen, and R. Deng. A survey of predictive maintenance: Systems, purposes and approaches. *arXiv preprint arXiv:1911.10539*, 2019.

[15] A. Saxena, K. Goebel, D. Simon, and N. Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. *International Conference on Prognostics and Health Management*, pages 1–9, 2008.

[16] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.

[17] X.-S. Si, W. Wang, C.-H. Hu, and D.-H. Zhou. Remaining useful life estimation–a review on the statistical data driven approaches. *European Journal of Operational Research*, 213(1):1–14, 2011.

[18] UWE Bristol. Aircraft predictive maintenance automation and optimisation: Synthesised landing gear datasets. UWE Bristol Research Data Repository, 2023.